

```

import { db } from "../firebase-config.js";
import {
  doc, getDoc, setDoc, updateDoc, getDocs, deleteField,
  collection, query, where, onSnapshot,
  addDoc, increment, arrayUnion, arrayRemove,
} from "https://www.gstatic.com/firebasejs/10.13.0/firebase-firestore.js";

// ----- crypto helpers -----
// NOTE: this is basic obfuscation of message text at rest in Firestore, not
// real end-to-end encryption – the "secret" is derived from the same chatId
// that is stored in plain sight on every message document, so anyone who can
// read the raw documents (e.g. from the Firebase console) can derive the same
// key. It stops a casual reader of the database from seeing plaintext, but it
// is NOT a security boundary against a motivated attacker. Don't advertise it
// as "encrypted" in the UI.
const sha256Hex = async (str) => {
  const buf = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(str))
  return Array.from(new Uint8Array(buf)).map((b) => b.toString(16).padStart(2, "0"))
};

const deriveAesKey = async (secret) => {
  const digest = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(secret))
  return crypto.subtle.importKey("raw", digest, { name: "AES-GCM" }, false, ["encrypt", "decrypt"]);
};

const encryptText = async (text, secret) => {
  const key = await deriveAesKey(secret);
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const cipherBuf = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, key, new TextEncoder().encode(text));
  return { iv: Array.from(iv), data: Array.from(new Uint8Array(cipherBuf)) };
};

const decryptText = async (payload, secret) => {
  if (!payload) return "";
  try {
    const key = await deriveAesKey(secret);
    const iv = new Uint8Array(payload.iv);
    const data = new Uint8Array(payload.data);
    const plainBuf = await crypto.subtle.decrypt({ name: "AES-GCM", iv }, key, new Uint8Array(data));
    return new TextDecoder().decode(plainBuf);
  } catch (e) {
    return "⚠️ تشفير فاش";
  }
};

const dmChatId = (a, b) => "dm__" + [a, b].sort().join("__");
const chatIdFor = (chat) => (chat.type === "dm" ? dmChatId(session.username, chat) : chat.id);

```

```

const secretForChat = (chat) => (chat.type === "dm" ? dmChatId(session.username,

// compress an uploaded image file into a small square JPEG data URL
const fileToAvatarDataURL = (file, maxSize = 128, quality = 0.75) =>
  new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onerror = reject;
    reader.onload = (e) => {
      const img = new Image();
      img.onerror = reject;
      img.onload = () => {
        const side = Math.min(img.width, img.height);
        const sx = (img.width - side) / 2;
        const sy = (img.height - side) / 2;
        const canvas = document.createElement("canvas");
        canvas.width = maxSize;
        canvas.height = maxSize;
        const ctx = canvas.getContext("2d");
        ctx.drawImage(img, sx, sy, side, side, 0, 0, maxSize, maxSize);
        resolve(canvas.toDataURL("image/jpeg", quality));
      };
      img.src = e.target.result;
    };
    reader.readAsDataURL(file);
  });

const escapeHtml = (s) =>
  String(s ?? "").replace(/[\&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "

// ----- state -----
let session = null; // {username, id, displayName, avatar}
let selectedChat = null; // {type:'dm'|"group", id, name}
let friendsUnsub = null;
let groupsUnsub = null;
let messagesUnsub = null;
let presenceUnsub = null;
let visitsUnsub = null;
let readsUnsub = null;
let typingUnsub = null;
let friendPresenceUnsub = null;
let pendingAvatarDataURL = null;
let forgotTargetUsername = null;
let presenceIntervalId = null;
let typingClearTimeoutId = null;

let editingMessageId = null; // set while editing an existing message

```

```

let replyTarget = null; // {id, from, preview} set while composing a reply
let currentReadsMap = {}; // { username: lastReadTs } for the open chat
let lastRenderedMessages = []; // decrypted list for the open chat, for re-render
let friendIsTyping = false;
let friendIsOnline = null; // null = n/a (group chats), true/false for dm

// ----- DOM refs -----
const $ = (id) => document.getElementById(id);
const authScreen = $("authScreen"), appScreen = $("appScreen");
const authUser = $("authUser"), authPass = $("authPass"), authError = $("authError");
const secQuestion = $("secQuestion"), secAnswer = $("secAnswer"), forgotLink = $("forgotLink");
const meName = $("meName"), meHandle = $("meHandle"), meId = $("meId"), avatarBtn = $("avatarBtn");
const totalVisitsEl = $("totalVisits"), onlineCountEl = $("onlineCount");
const friendsList = $("friendsList"), groupsList = $("groupsList");
const requestsSection = $("requestsSection"), requestsList = $("requestsList");
const addFriendInput = $("addFriendInput"), addFriendBtn = $("addFriendBtn");
const tabFriends = $("tabFriends"), tabGroups = $("tabGroups");
const friendsPanel = $("friendsPanel"), groupsPanel = $("groupsPanel");
const newGroupBtn = $("newGroupBtn"), newGroupModal = $("newGroupModal");
const newGroupName = $("newGroupName"), newGroupMembers = $("newGroupMembers"), c = $("newGroupMembers");
const settingsBtn = $("settingsBtn"), settingsModal = $("settingsModal");
const avatarPreview = $("avatarPreview"), choosePhotoBtn = $("choosePhotoBtn"), p = $("choosePhotoBtn");
const displayNameInput = $("displayNameInput"), emailInput = $("emailInput"), pro = $("emailInput");
const verifiedBadgePreview = $("verifiedBadgePreview"), verifiedStatusText = $("verifiedStatusText");
const redeemCodeInput = $("redeemCodeInput"), redeemBtn = $("redeemBtn"), redeemE = $("redeemBtn");
const adminBlock = $("adminBlock"), genCodeBtn = $("genCodeBtn"), genCodeResult = $("genCodeResult");
const ADMIN_USERNAME = "x1877";
const oldPass = $("oldPass"), newPass = $("newPass"), newPass2 = $("newPass2"), s = $("newPass2");
const forgotModal = $("forgotModal"), forgotStep1 = $("forgotStep1"), forgotStep2 = $("forgotStep2");
const forgotUser = $("forgotUser"), forgotError1 = $("forgotError1"), forgotFetch = $("forgotFetch");
const forgotQuestionText = $("forgotQuestionText"), forgotAnswer = $("forgotAnswer");
const forgotNewPass = $("forgotNewPass"), forgotNewPass2 = $("forgotNewPass2"), f = $("forgotNewPass2");
const logoutBtn = $("logoutBtn"), copyIdBtn = $("copyIdBtn");
const emptyState = $("emptyState"), chatWindow = $("chatWindow"), chatTitle = $("chatTitle");
const messagesBox = $("messagesBox"), msgInput = $("msgInput"), sendBtn = $("sendBtn");
const replyBar = $("replyBar"), replyBarLabel = $("replyBarLabel"), replyBarPreview = $("replyBarPreview");
const toast = $("toast");

const showToast = (t) => {
  toast.textContent = t;
  toast.classList.remove("hidden");
  setTimeout(() => toast.classList.add("hidden"), 2200);
};

const renderAvatarInto = (el, name, avatarDataUrl) => {
  if (avatarDataUrl) {

```

```

        el.innerHTML = "";
        const img = document.createElement("img");
        img.src = avatarDataUrl;
        el.appendChild(img);
    } else {
        el.textContent = name?.[0]?.toUpperCase() || "?";
    }
};

// ----- auth -----
authBtn.onclick = handleAuth;
authPass.addEventListener("keydown", (e) => e.key === "Enter" && handleAuth());
forgotLink.onclick = () => {
    forgotStep1.classList.remove("hidden");
    forgotStep2.classList.add("hidden");
    forgotUser.value = "";
    forgotAnswer.value = "";
    forgotNewPass.value = "";
    forgotNewPass2.value = "";
    forgotError1.classList.add("hidden");
    forgotError2.classList.add("hidden");
    forgotModal.classList.remove("hidden");
};

async function handleAuth() {
    const uname = authUser.value.trim();
    const pass = authPass.value;
    authError.classList.add("hidden");
    if (!uname || !pass) {
        authError.textContent = "× لازم یوزر و پاسورد";
        authError.classList.remove("hidden");
        return;
    }
    authBtn.disabled = true;
    authBtn.textContent = "...";
    try {
        const ref = doc(db, "chatUsers", uname);
        const snap = await getDoc(ref);
        const passHash = await sha256Hex(uname.toLowerCase() + ":" + pass);
        if (snap.exists()) {
            const data = snap.data();
            if (data.passHash !== passHash) {
                authError.textContent = "× رمز خاطی";
                authError.classList.remove("hidden");
                authBtn.disabled = false;
                authBtn.textContent = "> connect";
            }
        }
    }
}

```

```

        return;
    }
    session = {
        username: uname, id: data.id, displayName: data.displayName || uname,
        avatar: data.avatar || "", email: data.email || "", verified: !!data.veri
    };
} else {
    if (!secQuestion.value.trim() || !secAnswer.value.trim()) {
        authError.textContent = "× سؤال الأمان و جوابه (تحتاجهم لو نسيت الباسورد)";
        authError.classList.remove("hidden");
        authBtn.disabled = false;
        authBtn.textContent = "> connect";
        return;
    }
    const id = String(Math.floor(100000 + Math.random() * 899999));
    const secAnswerHash = await sha256Hex(uname.toLowerCase() + ":" + secAnswer);
    await setDoc(ref, {
        passHash, id, friends: [], incomingRequests: [], outgoingRequests: [], cr
        displayName: uname, avatar: "", email: "", verified: false,
        secQuestion: secQuestion.value.trim(), secAnswerHash,
    });
    // small lookup doc so "add friend by #id" doesn't require scanning the
    // whole chatUsers collection (see addFriend()).
    await setDoc(doc(db, "chatUserIds", id), { username: uname });
    session = { username: uname, id, displayName: uname, avatar: "", email: "",
        showToast("✓ تم إنشاء الحساب");
    }
    localStorage.setItem("x1877chat_session", JSON.stringify(session));
    enterApp();
} catch (e) {
    console.error(e);
    authError.textContent = "× صار خطأ بالاتصال، جرب مرة ثانية";
    authError.classList.remove("hidden");
}
authBtn.disabled = false;
authBtn.textContent = "> connect";
}

// restore session on load
(function restoreSession() {
    const raw = localStorage.getItem("x1877chat_session");
    if (raw) {
        try {
            session = JSON.parse(raw);
            enterApp();
        } catch (e) {}
    }
}

```

```

    }
  })();

function enterApp() {
  authScreen.classList.add("hidden");
  appScreen.classList.remove("hidden");
  refreshMeUI();
  bumpVisitorCount();
  startPresenceHeartbeat();
  listenFriends();
  listenGroups();
}

function refreshMeUI() {
  meName.innerHTML = "";
  meName.appendChild(document.createTextNode(session.displayName || session.usern
  if (session.verified) {
    const b = document.createElement("span");
    b.className = "badge";
    b.textContent = "✓";
    b.style.marginRight = "4px";
    meName.appendChild(b);
  }
  meHandle.textContent = "@" + session.username;
  meId.textContent = session.id;
  renderAvatarInto(avatarBtn, session.displayName || session.username, session.av
}

logoutBtn.onclick = () => {
  session = null;
  selectedChat = null;
  localStorage.removeItem("x1877chat_session");
  if (friendsUnsub) friendsUnsub();
  if (groupsUnsub) groupsUnsub();
  if (messagesUnsub) messagesUnsub();
  if (presenceUnsub) presenceUnsub();
  if (visitsUnsub) visitsUnsub();
  if (readsUnsub) readsUnsub();
  if (typingUnsub) typingUnsub();
  if (friendPresenceUnsub) friendPresenceUnsub();
  if (presenceIntervalId) { clearInterval(presenceIntervalId); presenceIntervalId
  if (typingClearTimeoutId) { clearTimeout(typingClearTimeoutId); typingClearTime
  authScreen.classList.remove("hidden");
  appScreen.classList.add("hidden");
  authUser.value = "";
  authPass.value = "";

```

```

};

copyIdBtn.onclick = () => {
  navigator.clipboard?.writeText(session.id);
  showToast("Copied ✓");
};

// ----- profile: avatar + display name -----
avatarBtn.onclick = openProfileModal;
settingsBtn.onclick = openProfileModal;
document.querySelectorAll("[data-close]").forEach((btn) => {
  btn.onclick = () => $(btn.dataset.close).classList.add("hidden");
});

function openProfileModal() {
  pendingAvatarDataUrl = null;
  displayNameInput.value = session.displayName || session.username;
  emailInput.value = session.email || "";
  renderAvatarInto(avatarPreview, session.displayName || session.username, session.email);
  profileError.classList.add("hidden");
  oldPass.value = ""; newPass.value = ""; newPass2.value = "";
  settingsError.classList.add("hidden");

  verifiedBadgePreview.style.display = session.verified ? "inline-flex" : "none";
  verifiedStatusText.textContent = session.verified ? "حسابك مب موثق بعد" : "ثق ✓";
  redeemBlock.classList.toggle("hidden", !!session.verified);
  redeemCodeInput.value = "";
  redeemError.classList.add("hidden");

  adminBlock.classList.toggle("hidden", session.username !== ADMIN_USERNAME);
  genCodeResult.textContent = "";

  settingsModal.classList.remove("hidden");
}

choosePhotoBtn.onclick = () => photoInput.click();
photoInput.onChange = async () => {
  const file = photoInput.files?.[0];
  if (!file) return;
  try {
    pendingAvatarDataUrl = await fileToAvatarDataUrl(file);
    renderAvatarInto(avatarPreview, session.displayName || session.username, pendingAvatarDataUrl);
  } catch (e) {
    console.error(e);
    showToast("ما كدرت أفتح هذي الصورة ✕");
  }
}

```

```

};

saveProfileBtn.onclick = async () => {
  profileError.classList.add("hidden");
  const name = displayNameInput.value.trim();
  if (!name) {
    profileError.textContent = "× حط اسم";
    profileError.classList.remove("hidden");
    return;
  }
  saveProfileBtn.disabled = true;
  const update = { displayName: name, email: emailInput.value.trim() };
  if (pendingAvatarDataUrl) update.avatar = pendingAvatarDataUrl;
  try {
    await updateDoc(doc(db, "chatUsers", session.username), update);
    session.displayName = name;
    session.email = emailInput.value.trim();
    if (pendingAvatarDataUrl) session.avatar = pendingAvatarDataUrl;
    localStorage.setItem("x1877chat_session", JSON.stringify(session));
    refreshMeUI();
    showToast("✓ تحفظ الملف الشخصي");
  } catch (e) {
    console.error(e);
    profileError.textContent = "× صار خطأ، جرب مرة ثانية (يمكن الصورة كبيرة)";
    profileError.classList.remove("hidden");
  }
  saveProfileBtn.disabled = false;
};

// ----- password change (while logged in) -----
changePassBtn.onclick = async () => {
  settingsError.classList.add("hidden");
  if (!oldPass.value || !newPass.value) {
    settingsError.textContent = "× عبي كل الحقول";
    settingsError.classList.remove("hidden");
    return;
  }
  if (newPass.value !== newPass2.value) {
    settingsError.textContent = "× الباسورد الجديد ما يتطابق";
    settingsError.classList.remove("hidden");
    return;
  }
  changePassBtn.disabled = true;
  const ref = doc(db, "chatUsers", session.username);
  const snap = await getDoc(ref);
  const data = snap.data();

```



```

const oldHash = await sha256Hex(session.username.toLowerCase() + ":" + oldPass.
if (oldHash !== data.passHash) {
  settingsError.textContent = "× الرمز الحالي غلط";
  settingsError.classList.remove("hidden");
  changePassBtn.disabled = false;
  return;
}
const newHash = await sha256Hex(session.username.toLowerCase() + ":" + newPass.
await updateDoc(ref, { passHash: newHash });
changePassBtn.disabled = false;
settingsModal.classList.add("hidden");
oldPass.value = ""; newPass.value = ""; newPass2.value = "";
showToast("✓ تغيّر الرمز");
};

// ----- verified: redeem code -----
redeemBtn.onclick = async () => {
  redeemError.classList.add("hidden");
  const code = redeemCodeInput.value.trim().toUpperCase();
  if (!code) {
    redeemError.textContent = "× حظ الكود";
    redeemError.classList.remove("hidden");
    return;
  }
  redeemBtn.disabled = true;
  try {
    const codeRef = doc(db, "premiumCodes", code);
    const snap = await getDoc(codeRef);
    if (!snap.exists() || snap.data().used) {
      redeemError.textContent = "× الكود غلط أو مستخدم من قبل";
      redeemError.classList.remove("hidden");
      redeemBtn.disabled = false;
      return;
    }
    await updateDoc(codeRef, { used: true, usedBy: session.username });
    await updateDoc(doc(db, "chatUsers", session.username), { verified: true });
    session.verified = true;
    localStorage.setItem("x1877chat_session", JSON.stringify(session));
    refreshMeUI();
    verifiedBadgePreview.style.display = "inline-flex";
    verifiedStatusText.textContent = "✓ حسابك موثّق";
    redeemBlock.classList.add("hidden");
    showToast("✓ مبروك، حسابك صار موثّق");
  } catch (e) {
    console.error(e);
    redeemError.textContent = "× صار خطأ، جرب مرة ثانية";
  }
}

```

```

        redeemError.classList.remove("hidden");
    }
    redeemBtn.disabled = false;
};

// ----- admin: generate premium codes (x1877 only) -----
// NOTE: this check is purely client-side UI – it hides/shows a button, it does
// NOT protect the "premiumCodes" collection. Without Firebase Auth + Firestore
// security rules that verify identity server-side, anyone who edits their own
// localStorage session (or calls Firestore directly) can still write codes.
// Locking this down for real requires the Auth work that was intentionally
// skipped for now – treat this button as convenience, not a security boundary.
genCodeBtn.onclick = async () => {
    if (session.username !== ADMIN_USERNAME) return;
    const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
    const rnd = () => Array.from(crypto.getRandomValues(new Uint8Array(4))).map((b)
    const code = `${rnd()}-${rnd()}-${rnd()}`;
    genCodeBtn.disabled = true;
    try {
        await setDoc(doc(db, "premiumCodes", code), { used: false, createdAt: Date.no
        genCodeResult.textContent = "الكود: " + code;
        navigator.clipboard?.writeText(code);
        showToast("✓ انسخ الكود");
    } catch (e) {
        console.error(e);
        genCodeResult.textContent = "× مار خطأ";
    }
    genCodeBtn.disabled = false;
};

// ----- forgot password (security question) -----
forgotFetchBtn.onclick = async () => {
    forgotError1.classList.add("hidden");
    const uname = forgotUser.value.trim();
    if (!uname) {
        forgotError1.textContent = "× اكتب اليوزر";
        forgotError1.classList.remove("hidden");
        return;
    }
    forgotFetchBtn.disabled = true;
    const snap = await getDoc(doc(db, "chatUsers", uname));
    forgotFetchBtn.disabled = false;
    if (!snap.exists() || !snap.data().secQuestion) {
        forgotError1.textContent = "× ما لكيت هذا اليوزر أو ما عنده سؤال أمان محفوظ";
        forgotError1.classList.remove("hidden");
        return;
    }

```

```

    }
    forgotTargetUsername = uname;
    forgotQuestionText.textContent = snap.data().secQuestion;
    forgotStep1.classList.add("hidden");
    forgotStep2.classList.remove("hidden");
  };

forgotResetBtn.onclick = async () => {
  forgotError2.classList.add("hidden");
  if (!forgotAnswer.value.trim() || !forgotNewPass.value) {
    forgotError2.textContent = "× عبي كل الحقول";
    forgotError2.classList.remove("hidden");
    return;
  }
  if (forgotNewPass.value !== forgotNewPass2.value) {
    forgotError2.textContent = "× الباسورد الجديد ما يتطابق";
    forgotError2.classList.remove("hidden");
    return;
  }
  forgotResetBtn.disabled = true;
  const ref = doc(db, "chatUsers", forgotTargetUsername);
  const snap = await getDoc(ref);
  const data = snap.data();
  const answerHash = await sha256Hex(forgotTargetUsername.toLowerCase() + ":" + f
  if (answerHash !== data.secAnswerHash) {
    forgotError2.textContent = "× الجواب غلط";
    forgotError2.classList.remove("hidden");
    forgotResetBtn.disabled = false;
    return;
  }
  const newHash = await sha256Hex(forgotTargetUsername.toLowerCase() + ":" + forg
  await updateDoc(ref, { passHash: newHash });
  forgotResetBtn.disabled = false;
  forgotModal.classList.add("hidden");
  showToast("تغير الباسورد ✓ سجل دخول الجين");
};

// ----- visitor / presence counters -----
async function bumpVisitorCount() {
  const ref = doc(db, "chatStats", "counters");
  try {
    await setDoc(ref, { totalVisits: increment(1) }, { merge: true });
  } catch (e) { console.error(e); }
  if (visitsUnsub) visitsUnsub(); // avoid stacking listeners across repeated log
  visitsUnsub = onSnapshot(ref, (snap) => {
    totalVisitsEl.textContent = snap.exists() ? (snap.data().totalVisits ?? 0) :

```

```

    });
}

function startPresenceHeartbeat() {
    const ref = doc(db, "chatPresence", session.username);
    const beat = () => setDoc(ref, { ts: Date.now() }).catch(() => {});
    beat();
    if (presenceIntervalId) clearInterval(presenceIntervalId); // fixes: old interv
    presenceIntervalId = setInterval(beat, 10000);

    if (presenceUnsub) presenceUnsub();
    presenceUnsub = onSnapshot(collection(db, "chatPresence"), (snap) => {
        let online = 0;
        snap.forEach((d) => {
            const ts = d.data().ts;
            if (ts && Date.now() - ts < 25000) online++;
        });
        onlineCountEl.textContent = Math.max(online, 1);
    });
}

// ----- friends -----
tabFriends.onclick = () => switchTab("friends");
tabGroups.onclick = () => switchTab("groups");
function switchTab(tab) {
    tabFriends.classList.toggle("active", tab === "friends");
    tabGroups.classList.toggle("active", tab === "groups");
    friendsPanel.classList.toggle("hidden", tab !== "friends");
    groupsPanel.classList.toggle("hidden", tab !== "groups");
}

function listenFriends() {
    const ref = doc(db, "chatUsers", session.username);
    if (friendsUnsub) friendsUnsub();
    friendsUnsub = onSnapshot(
        ref,
        async (snap) => {
            if (!snap.exists()) return;
            const data = snap.data();
            const usernames = data.friends || [];
            const incoming = data.incomingRequests || [];
            const outgoing = data.outgoingRequests || [];

            const objs = await Promise.all(
                usernames.map(async (f) => {
                    const s = await getDoc(doc(db, "chatUsers", f));

```

```

        return s.exists()
        ? { username: f, id: s.data().id, displayName: s.data().displayName |
          : { username: f, id: "?", displayName: f, avatar: "", verified: false
        })
    );
    const incomingObjs = await Promise.all(
        incoming.map(async (f) => {
            const s = await getDoc(doc(db, "chatUsers", f));
            return { username: f, displayName: s.exists() ? (s.data().displayName |
            })
        })
    );

    renderFriends(objs, outgoing);
    renderRequests(incomingObjs);
    renderGroupMemberChecklist(objs);
},
(err) => {
    console.error("friends listener error", err);
    showToast("× بجلب الأصدقاء : خطأ " + err.message);
}
);
}

```

```

function renderRequests(incomingObjs) {
    requestsList.innerHTML = "";
    if (incomingObjs.length === 0) {
        requestsSection.classList.add("hidden");
        return;
    }
    requestsSection.classList.remove("hidden");
    incomingObjs.forEach((r) => {
        const row = document.createElement("div");
        row.className = "friendReqRow";
        const nameWrap = document.createElement("div");
        nameWrap.className = "friendReqName";
        nameWrap.appendChild(document.createTextNode(r.displayName));
        const actions = document.createElement("div");
        actions.className = "friendReqActions";
        const acceptBtn = document.createElement("button");
        acceptBtn.className = "tinyBtn accept";
        acceptBtn.textContent = "قبول";
        acceptBtn.onclick = () => acceptRequest(r.username);
        const declineBtn = document.createElement("button");
        declineBtn.className = "tinyBtn decline";
        declineBtn.textContent = "رفض";
        declineBtn.onclick = () => declineRequest(r.username);
    });
}

```

```

        actions.appendChild(acceptBtn);
        actions.appendChild(declineBtn);
        row.appendChild(nameWrap);
        row.appendChild(actions);
        requestsList.appendChild(row);
    });
}

```

```

async function acceptRequest(fromUsername) {
    try {
        await updateDoc(doc(db, "chatUsers", session.username), {
            incomingRequests: arrayRemove(fromUsername),
            friends: arrayUnion(fromUsername),
        });
        await updateDoc(doc(db, "chatUsers", fromUsername), {
            outgoingRequests: arrayRemove(session.username),
            friends: arrayUnion(session.username),
        });
        showToast("صار صديقك ✓");
    } catch (e) {
        console.error(e);
        showToast("صار خطأ ✕");
    }
}

```

```

async function declineRequest(fromUsername) {
    try {
        await updateDoc(doc(db, "chatUsers", session.username), { incomingRequests: a
        await updateDoc(doc(db, "chatUsers", fromUsername), { outgoingRequests: array
        showToast("تم رفض الطلب");
    } catch (e) {
        console.error(e);
        showToast("صار خطأ ✕");
    }
}

```

```

function renderFriends(objs, outgoing) {
    friendsList.innerHTML = "";
    if (objs.length === 0 && (!outgoing || outgoing.length === 0)) {
        friendsList.innerHTML = `<p class="emptyHint">بعد أصدقاء بعد</p>`;
        return;
    }
    objs.forEach((f) => {
        const btn = document.createElement("button");
        btn.className = "listItem" + (selectedChat?.id === f.username && selectedChat
        const dot = document.createElement("span");

```

```

    dot.className = "dot";
    btn.appendChild(dot);
    btn.appendChild(document.createTextNode(f.displayName));
    if (f.verified) {
        const b = document.createElement("span");
        b.className = "badge";
        b.textContent = "✓";
        btn.appendChild(b);
    }
    btn.onclick = () => openChat({ type: "dm", id: f.username, name: f.displayName});
    friendsList.appendChild(btn);
});
(outgoing || []).forEach((u) => {
    const row = document.createElement("div");
    row.className = "listItem";
    row.style.cursor = "default";
    row.innerHTML = `<span class="dot offline"></span>${escapeHtml(u)}<span class="
    friendsList.appendChild(row);
});
}

```

```

addFriendBtn.onclick = addFriend;

```

```

addFriendInput.addEventListener("keydown", (e) => e.key === "Enter" && addFriend(

```

```

async function addFriend() {
    const q = addFriendInput.value.trim();
    if (!q) return;
    if (q === session.username) return showToast("ما تكدر تضيف نفسك");

    let targetUsername = q;
    let targetSnap = await getDoc(doc(db, "chatUsers", q));
    if (!targetSnap.exists()) {
        // look up by #id via a small dedicated doc instead of scanning the whole
        // chatUsers collection (which used to expose every user's data to any
        // client just to resolve one ID).
        const idSnap = await getDoc(doc(db, "chatUserIds", q));
        if (!idSnap.exists()) return showToast("اليوزر هذا اليوزر");
        targetUsername = idSnap.data().username;
        targetSnap = await getDoc(doc(db, "chatUsers", targetUsername));
        if (!targetSnap.exists()) return showToast("اليوزر هذا اليوزر");
    }
}

```

```

const meRef = doc(db, "chatUsers", session.username);
const meSnap = await getDoc(meRef);
const meData = meSnap.data();

```

```

    if ((meData.friends || []).includes(targetUsername)) return showToast("ديكك خلص");
    if ((meData.outgoingRequests || []).includes(targetUsername)) return showToast(

// if they already sent me a request, accept it instead of creating a duplicate
    if ((meData.incomingRequests || []).includes(targetUsername)) {
        await acceptRequest(targetUsername);
        addFriendInput.value = "";
        return;
    }

    await updateDoc(meRef, { outgoingRequests: arrayUnion(targetUsername) });
    await updateDoc(doc(db, "chatUsers", targetUsername), { incomingRequests: array
    addFriendInput.value = "";
    showToast("تم إرسال طلب المداقة");
}

// ----- groups -----
function listenGroups() {
    const q = query(collection(db, "chatGroups"), where("members", "array-contains"
    if (groupsUnsub) groupsUnsub();
    groupsUnsub = onSnapshot(
        q,
        (snap) => {
            const groups = [];
            snap.forEach((d) => groups.push({ id: d.id, ...d.data() }));
            renderGroups(groups);
        },
        (err) => {
            console.error("groups listener error", err);
            showToast("خطأ بجلب الكروبات " + err.message);
        }
    );
}

function renderGroups(groups) {
    groupsList.innerHTML = "";
    if (groups.length === 0) {
        groupsList.innerHTML = `<p class="emptyHint">عندك كروبات بعد</p>`;
        return;
    }
    groups.forEach((g) => {
        const btn = document.createElement("button");
        btn.className = "listItem" + (selectedChat?.id === g.id && selectedChat?.type
        btn.innerHTML = `# ${escapeHtml(g.name)}<span class="groupMeta">${(g.members
        btn.onclick = () => openChat({ type: "group", id: g.id, name: g.name, memberC
        groupsList.appendChild(btn);
    });
}

```



```

    });
}

let selectedFriendsForGroup = new Set();
newGroupBtn.onclick = () => {
    newGroupName.value = "";
    selectedFriendsForGroup = new Set();
    newGroupModal.classList.remove("hidden");
};

function renderGroupMemberChecklist(friendObjs) {
    newGroupMembers.innerHTML = "";
    if (friendObjs.length === 0) {
        newGroupMembers.innerHTML = `<p class="emptyHint">أول أصدقاء أضيف</p>`;
        return;
    }
    friendObjs.forEach((f) => {
        const label = document.createElement("label");
        const cb = document.createElement("input");
        cb.type = "checkbox";
        cb.onchange = () => {
            if (cb.checked) selectedFriendsForGroup.add(f.username);
            else selectedFriendsForGroup.delete(f.username);
        };
        label.appendChild(cb);
        label.appendChild(document.createTextNode(f.displayName));
        newGroupMembers.appendChild(label);
    });
}

createGroupBtn.onclick = async () => {
    const name = newGroupName.value.trim();
    if (!name) return;
    const members = Array.from(new Set([session.username, ...selectedFriendsForGroup]));
    await addDoc(collection(db, "chatGroups"), {
        name, members, owner: session.username, createdAt: Date.now(),
    });
    newGroupModal.classList.add("hidden");
    showToast("تم إنشاء الكروب");
};

// ----- chat / messages -----
function openChat(chat) {
    selectedChat = chat;
    editingMessageId = null;
    replyTarget = null;
}

```

```

hideComposerBar();
emptyState.classList.add("hidden");
chatWindow.classList.remove("hidden");
chatTitle.textContent = (chat.type === "group" ? "#" : "@") + chat.name;
messagesBox.innerHTML = "";
currentReadsMap = {};
document.querySelectorAll("#friendsList .listItem, #groupsList .listItem").forEach

if (readsUnsub) readsUnsub();
if (typingUnsub) typingUnsub();
if (friendPresenceUnsub) friendPresenceUnsub();

if (chat.type === "group") {
    friendIsOnline = null;
    chatSubStatus.textContent = `${chat.memberCount ?? ""} أعضاء`.trim();
    chatSubStatus.classList.remove("typing");
} else {
    chatSubStatus.textContent = "";
    listenFriendPresence(chat.id);
}

listenMessages();
listenReads();
listenTyping();
}

function listenFriendPresence(otherUsername) {
    friendPresenceUnsub = onSnapshot(doc(db, "chatPresence", otherUsername), (snap)
        const ts = snap.exists() ? snap.data().ts : 0;
        friendIsOnline = !!ts && Date.now() - ts < 25000;
        renderChatSubStatus();
    });
}

function renderChatSubStatus() {
    if (!selectedChat) return;
    if (selectedChat.type === "group") return; // static member count already set
    if (friendIsTyping) {
        chatSubStatus.textContent = "يكتب...";
        chatSubStatus.classList.add("typing");
    } else {
        chatSubStatus.classList.remove("typing");
        chatSubStatus.textContent = friendIsOnline ? "متصل الآن" : "غير متصل";
    }
}

```

```

function listenReads() {
  const chatId = chatIdFor(selectedChat);
  markChatRead(chatId);
  readsUnsub = onSnapshot(doc(db, "chatReads", chatId), (snap) => {
    currentReadsMap = snap.exists() ? snap.data() : {};
    renderMessages(lastRenderedMessages);
  });
}

function markChatRead(chatId) {
  setDoc(doc(db, "chatReads", chatId), { [session.username]: Date.now() }, { merge: true })
}

function listenTyping() {
  const chatId = chatIdFor(selectedChat);
  typingUnsub = onSnapshot(doc(db, "chatTyping", chatId), (snap) => {
    const data = snap.exists() ? snap.data() : {};
    const now = Date.now();
    friendIsTyping = Object.entries(data).some(([user, ts]) => user !== session.username)
    renderChatSubStatus();
  });
}

let lastTypingSentAt = 0;
msgInput.addEventListener("input", () => {
  if (!selectedChat) return;
  const now = Date.now();
  if (now - lastTypingSentAt > 1500) {
    lastTypingSentAt = now;
    const chatId = chatIdFor(selectedChat);
    setDoc(doc(db, "chatTyping", chatId), { [session.username]: now }, { merge: true })
  }
  if (typingClearTimeoutId) clearTimeout(typingClearTimeoutId);
  typingClearTimeoutId = setTimeout(() => {
    if (!selectedChat) return;
    const chatId = chatIdFor(selectedChat);
    setDoc(doc(db, "chatTyping", chatId), { [session.username]: 0 }, { merge: true }, 3000);
  });
});

function listenMessages() {
  const chatId = chatIdFor(selectedChat);
  const secret = secretForChat(selectedChat);
  // NOTE: no orderBy here on purpose - where() + orderBy() on a different field
  // needs a composite Firestore index, and without it the listener fails silently
  // We sort by timestamp ourselves after fetching instead.

```

```

const q = query(collection(db, "chatMessages"), where("chatId", "==", chatId));
if (messagesUnsub) messagesUnsub();
messagesUnsub = onSnapshot(
  q,
  async (snap) => {
    const raw = [];
    snap.forEach((d) => raw.push({ id: d.id, ...d.data() }));
    raw.sort((a, b) => a.ts - b.ts);
    const decrypted = await Promise.all(
      raw.map(async (m) => {
        const base = {
          id: m.id, from: m.from, ts: m.ts, sticker: !!m.sticker,
          edited: !!m.edited, deleted: !!m.deleted,
        };
        if (m.deleted) return { ...base, text: "" };
        base.text = await decryptText(m.enc, secret);
        if (m.replyTo) {
          base.replyTo = {
            id: m.replyTo.id, from: m.replyTo.from,
            preview: await decryptText(m.replyTo.encPreview, secret),
          };
        }
        return base;
      })
    );
    lastRenderedMessages = decrypted;
    renderMessages(decrypted);
    if (selectedChat) markChatRead(chatIdFor(selectedChat));
  },
  (err) => {
    console.error("messages listener error", err);
    showToast("ما كدرت أجيب الرسائل ✕: " + err.message);
  }
);
}

function renderMessages(list) {
  messagesBox.innerHTML = "";
  if (list.length === 0) {
    messagesBox.innerHTML = `
```

```

const mine = m.from === session.username;
const row = document.createElement("div");
row.className = "bubbleRow " + (mine ? "mine" : "theirs");

const bubble = document.createElement("div");
bubble.className = (m.sticker ? "bubble sticker" : "bubble ") + (mine ? "mine" : "theirs");

if (selectedChat.type === "group" && !mine && !m.sticker) {
    const author = document.createElement("p");
    author.className = "bubbleAuthor";
    author.textContent = m.from;
    bubble.appendChild(author);
}

if (m.replyTo && !m.deleted) {
    const quote = document.createElement("div");
    quote.className = "replyQuote";
    const author = document.createElement("span");
    author.className = "replyQuoteAuthor";
    author.textContent = m.replyTo.from;
    quote.appendChild(author);
    quote.appendChild(document.createTextNode(m.replyTo.preview));
    bubble.appendChild(quote);
}

const text = document.createElement("p");
text.className = m.sticker ? "stickerBubble" : "bubbleText";
text.textContent = m.deleted ? "تم حذف هذه الرسالة" : m.text;
bubble.appendChild(text);

if (!m.sticker) {
    const meta = document.createElement("div");
    meta.className = "bubbleMeta";
    if (m.edited && !m.deleted) {
        const tag = document.createElement("span");
        tag.className = "editedTag";
        tag.textContent = "(معدلة)";
        meta.appendChild(tag);
    }
    if (mine && otherUsername && !m.deleted) {
        const ticks = document.createElement("span");
        const seen = otherReadTs && m.ts <= otherReadTs;
        ticks.className = "seenTicks" + (seen ? " seen" : "");
        ticks.textContent = seen ? "✓✓" : "✓";
        meta.appendChild(ticks);
    }
}

```

```

        if (meta.childNodes.length) bubble.appendChild(meta);
    }

    if (!m.deleted) {
        const actions = document.createElement("div");
        actions.className = "msgActions";
        const replyBtn = document.createElement("button");
        replyBtn.className = "msgActionBtn";
        replyBtn.type = "button";
        replyBtn.textContent = "رد";
        replyBtn.onclick = () => startReply(m);
        actions.appendChild(replyBtn);
        if (mine && !m.sticker) {
            const editBtn = document.createElement("button");
            editBtn.className = "msgActionBtn";
            editBtn.type = "button";
            editBtn.textContent = "تعديل";
            editBtn.onclick = () => startEdit(m);
            actions.appendChild(editBtn);
            const delBtn = document.createElement("button");
            delBtn.className = "msgActionBtn danger";
            delBtn.type = "button";
            delBtn.textContent = "حذف";
            delBtn.onclick = () => deleteMessage(m);
            actions.appendChild(delBtn);
        }
        row.appendChild(bubble);
        row.appendChild(actions);
    } else {
        row.appendChild(bubble);
    }

    messagesBox.appendChild(row);
});
messagesBox.scrollTop = messagesBox.scrollHeight;
}

function startReply(m) {
    editingMessageId = null;
    replyTarget = { id: m.id, from: m.from, preview: m.text.slice(0, 60) };
    replyBar.classList.remove("hidden", "editingBar");
    replyBarLabel.textContent = "الرد على " + m.from + ":";
    replyBarPreview.textContent = replyTarget.preview;
    msgInput.focus();
}

```

```
function startEdit(m) {
  replyTarget = null;
  editingMessageId = m.id;
  replyBar.classList.remove("hidden");
  replyBar.classList.add("editingBar");
  replyBarLabel.textContent = "تعديل الرسالة:";
  replyBarPreview.textContent = m.text.slice(0, 60);
  msgInput.value = m.text;
  msgInput.focus();
}

function hideComposerBar() {
  replyBar.classList.add("hidden");
  replyBar.classList.remove("editingBar");
  replyBarLabel.textContent = "";
  replyBarPreview.textContent = "";
}

replyBarClose.onclick = () => {
  editingMessageId = null;
  replyTarget = null;
  msgInput.value = "";
  hideComposerBar();
};

async function deleteMessage(m) {
  if (!confirm("تحذف هذي الرسالة؟")) return;
  try {
    await updateDoc(doc(db, "chatMessages", m.id), { deleted: true, enc: null, re
  } catch (e) {
    console.error(e);
    showToast("ما كدرت أحذف الرسالة ✕");
  }
}

sendBtn.onclick = sendMessage;
msgInput.addEventListener("keydown", (e) => e.key === "Enter" && sendMessage());

// ----- emoji + stickers -----
const EMOJIS = ["😄", "😂", "😭", "😘", "😍", "😘", "😎", "😏", "😭", "😭", "😭", "😡", "😭", "👍", "👍"];
const STICKERS = ["🎉", "😂", "❤️", "🔥", "👍", "😭", "😭", "😭", "👉", "👉", "👉", "👉", "👉", "👉"];
const emojiBtn = $("emojiBtn"), emojiPanel = $("emojiPanel");
const emojiGrid = $("emojiGrid"), stickerGrid = $("stickerGrid");
const emojiTabBtn = $("emojiTabBtn"), stickerTabBtn = $("stickerTabBtn");

EMOJIS.forEach((e) => {
```

```

    const b = document.createElement("button");
    b.type = "button";
    b.textContent = e;
    b.onclick = () => {
        msgInput.value += e;
        msgInput.focus();
    };
    emojiGrid.appendChild(b);
});
STICKERS.forEach((s) => {
    const b = document.createElement("button");
    b.type = "button";
    b.className = "stickerBubble";
    b.textContent = s;
    b.onclick = () => sendSticker(s);
    stickerGrid.appendChild(b);
});
emojiBtn.onclick = () => emojiPanel.classList.toggle("hidden");
emojiTabBtn.onclick = () => {
    emojiTabBtn.classList.add("active");
    stickerTabBtn.classList.remove("active");
    emojiGrid.classList.remove("hidden");
    stickerGrid.classList.add("hidden");
};
stickerTabBtn.onclick = () => {
    stickerTabBtn.classList.add("active");
    emojiTabBtn.classList.remove("active");
    stickerGrid.classList.remove("hidden");
    emojiGrid.classList.add("hidden");
};

async function sendSticker(sticker) {
    if (!selectedChat) return;
    const secret = secretForChat(selectedChat);
    const enc = await encryptText(sticker, secret);
    const chatId = chatIdFor(selectedChat);
    await addDoc(collection(db, "chatMessages"), { chatId, from: session.username,
    emojiPanel.classList.add("hidden");
}

async function sendMessage() {
    const text = msgInput.value.trim();
    if (!text || !selectedChat) return;
    const secret = secretForChat(selectedChat);

    if (editingMessageId) {

```



```

    const id = editingMessageId;
    msgInput.value = "";
    editingMessageId = null;
    hideComposerBar();
    try {
        const enc = await encryptText(text, secret);
        await updateDoc(doc(db, "chatMessages", id), { enc, edited: true });
    } catch (e) {
        console.error(e);
        showToast("× أَعْذَل الرِّسَالَة");
    }
    return;
}

msgInput.value = "";
const enc = await encryptText(text, secret);
const chatId = chatIdFor(selectedChat);
const payload = { chatId, from: session.username, enc, ts: Date.now() };
if (replyTarget) {
    payload.replyTo = {
        id: replyTarget.id, from: replyTarget.from,
        encPreview: await encryptText(replyTarget.preview, secret),
    };
    replyTarget = null;
    hideComposerBar();
}
await addDoc(collection(db, "chatMessages"), payload);
// clear my own typing flag right after sending
setDoc(doc(db, "chatTyping", chatId), { [session.username]: 0 }, { merge: true
}

```